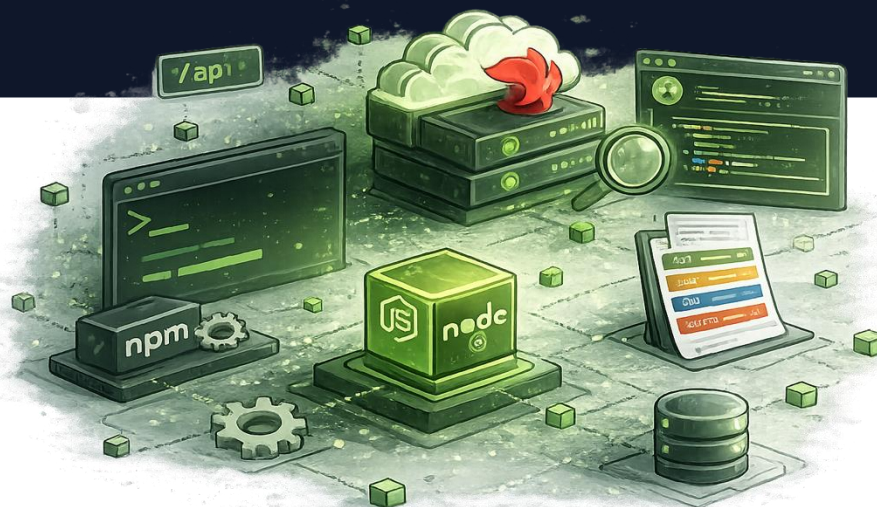




Progettazione di applicazioni web full stack

Laboratorio di Node e NestJS



Prof. Devis Bianchini

Dip. di Università degli Studi di Brescia



Argomento di questa lezione...

Cosa faremo

- Creazione di una libreria in **NestJS**
- Uso del pattern MVC in **NestJS** dentro **Nx**
- Utilizzo del meccanismo del «*routing*» in **NestJS**
- Approfondire la conoscenza di **Swagger**
- Comprendere i meccanismi di «*validazione*»

Output

- Una libreria backend in **node** utilizzando **NestJS** e **Nx**
- Pubblicazione delle REST API della libreria in **NestJS** tramite **Swagger**
- Implementazione dei meccanismi di «*routing*» e «*validazione*»

Dalle lezioni precedenti – Creare una libreria



```
npx nx g @nx/nest:library libs/server/users --controller --service
```

- Crea una nuova libreria basta su **NestJS** e **node**, che include anche un *servizio* ed un *controller*
- È anche possibile aggiungere l'opzione **--importPath** per assegnare un nome univoco alla libreria (per esempio, **--importPath=@server/users**)
- Le librerie sono riutilizzabili nelle app (create con il comando visto nella lezione L2 su **Nx**)

```
npx nx g @nx/nest:app apps/{appName}
```

```
npx nx g @nx/nest:library libs/server/users  
npx nx g @nx/nest:controller /libs/server/src/lib/users  
npx nx g @nx/nest:service /libs/server/src/lib/users
```

Controller della libreria



- File `users.controller.ts`

Il metodo `getUsers` restituisce un insieme di oggetti JSON:
- Il tipo `any []` indica che non viene imposto un vincolo particolare sul tipo restituito, a parte il fatto che deve trattarsi di un array

```
import { Controller, Get } from '@nestjs/common';
import { ApiTags } from '@nestjs/swagger';

@ApiTags('Users APIs')
@Controller('users')
export class UsersController {
  @Get()
  getUsers(): any[] {
    return [{ 'name': 'Mario Bianchi' },
            { 'name': 'Carlo Rossi' }];
  }
}
```

Creazione di librerie con NestJS e node



- Connettere i moduli `app.module.ts` e `users.module.ts`

Nel file `users.module.ts`:

- Vengono importati controller e service
- Il modulo viene esportato come `ServerUsersModule`

```
import { Module } from '@nestjs/common';
import { UsersController } from './users.controller';
import { UsersService } from './users.service';

@Module({
  controllers: [UsersController],
  providers: [UsersService],
  exports: [],
})
export class ServerUsersModule {}
```

Nel file `app.module.ts`:

- viene importato `ServerUsersModule`

```
import { ServerUsersModule } from '@server/users';

@Module({
  imports: [ServerUsersModule],
```

Meccanismo di routing



- Per controllare quale controller (e quale metodo all'interno del controller) è responsabile della gestione di una determinata richiesta HTTP

```
@ApiTags('Users APIs')
@Controller('users')
export class UsersController {
  /*
   GET /users
   GET /users/:id
  */

  @Get() // GET /users
  getUsers() {
    return [];
  }

  @Get('/:id') // GET /users/:id
  getOneUser(@Param('id') id: string) {
    return {id};
  }
}
```

Meccanismo di routing



- Attenzione: ogni parte di una rotta è una stringa
- L'ordine è importante

Proviamo a verificare il funzionamento delle due rotte:

- La seconda non viene mai invocata
- Perché?
- Che succede se invertiamo l'ordine dei due metodi (e rispettive rotte)?

```
@Get('/:id') // GET /users/:id
getOneUser(@Param('id') id: string) {
  return {id};
}

@Get('interns') // GET /users/interns
getInternUsers() {
  return []
}
```

Meccanismo di routing



- Nel caso di rotte che richiedono in input il body della richiesta HTTP, è possibile definire lo schema del body per poter testare la rotta con Swagger
- Il metodo più generale (che vedremo più avanti) è usare un **DTO (Data Transfer Object)**

```
@Post() // POST /users
@ApiBody({
  schema: {
    type: 'object',
    properties: {
      name: { type: 'string', example: 'Devis' },
      email: { type: 'string', example: 'bianchin@unibs.it' },
    },
    required: ['name', 'email'],
  },
})
create(@Body() user: {}) {
  return user;
}
```


Meccanismo di routing



- Nel caso di rotte che richiedono in input il body della richiesta HTTP, è possibile definire lo schema del body per poter testare la rotta con Swagger
- Il metodo più generale (che vedremo più avanti) è usare un **DTO (Data Transfer Object)**
- Analoga la rotta Patch, che combina diversi input

```
@Patch('/:id') // PATCH /users/:id
@ApiBody({
  schema: {
    type: 'object',
    properties: {
      name: { type: 'string', example: 'Devis' },
      email: { type: 'string', example: 'bianchin@unibs.it' },
    },
    required: ['name', 'email'],
  },
})
update(@Param('id') id: string, @Body() userUpdate: {}) {
  return {id, ...userUpdate};
}
```

Meccanismo di routing



- Nella definizione delle rotte, è anche possibile utilizzare le «*query string*»

```
@Get() // GET /users or /users?role=value
getUsers(@Query('role') role?: 'USER' | 'ADMIN') {
  return [];
}
```

Meccanismo dell'«injection»



- File `users.controller.ts`

- Si fa uso del costruttore del controller per «iniettare» il servizio (**provider**) in quest'ultimo
- Il servizio va importato normalmente

```
@ApiTags('Users APIs')
@Controller('users')
export class UsersController {
  constructor(private readonly userService: UsersService) {}

  @Get() // GET /users or /users?role=value
  getUsers(@Query('role') role?: 'USER' | 'ADMIN') {
    return this.userService.getUsers(role);
  }
}
```



- Importate dalla libreria '@nestjs/common' nel controller `users.controller.ts`

- Utilizzate per:
 - Transformation
 - Validation
- Rappresentano un tipo particolare di Middleware

```
@Get('/:id') // GET /users/:id
getOneUser(@Param('id') id: string) {
  return this.userService.getOneUser(+id);
}
```



```
@Get('/:id') // GET /users/:id
getOneUser(@Param('id', ParseIntPipe) id: number) {
  return this.userService.getOneUser(id);
}
```

Pipes



- Nella documentazione: docs.nestjs.com/pipes

INTRODUCTION

OVERVIEW

- First steps
- Controllers
- Providers
- Modules
- Middleware
- Exception filters
- Pipes**
- Guards
- Interceptors
- Custom decorators

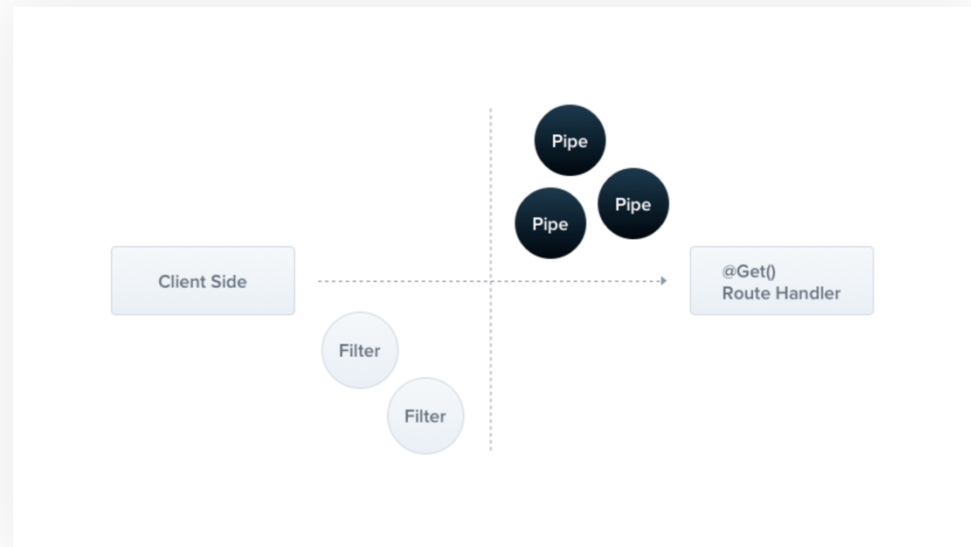
Built-in pipes

Nest comes with several pipes available out-of-the-box:

- `ValidationPipe`
- `ParseIntPipe`
- `ParseFloatPipe`
- `ParseBoolPipe`
- `ParseArrayPipe`
- `ParseUUIDPipe`
- `ParseEnumPipe`
- `DefaultValuePipe`
- `ParseFilePipe`
- `ParseDatePipe`

Pipes

A pipe is a class annotated with the `@Injectable()` decorator, which implements the `PipeTransform` interface.



Data Transfer Objects - DTO



- File `dto/create-user.dto.ts` e `dto/update-user.dto.ts` nei sorgenti

- Prevedono la definizione di uno «schema» per gli input di tipo object delle API

Può dare errore in fase di compilazione per una configurazione di default di TS (vincolo di inizializzazione)

Risolvibile mettendo `'strict': false` nel file `tsconfig.lib.json` del workspace

```
export enum UserRole {  
  USER = 'USER',  
  ADMIN = 'ADMIN'  
}  
  
export class CreateUserDto {  
  name: string;  
  email: string;  
  role: UserRole;  
};  
  
import { CreateUserDto } from "../create-user.dto";  
import { PartialType } from "@nestjs/mapped-types";  
  
export class UpdateUserDto extends PartialType(CreateUserDto) {  
}
```

Aggiunta di «class validators»



- File `dto/create-user.dto.ts` e `dto/update-user.dto.ts` nei sorgenti

- Per aggiungere funzionalità di validazione avanzate
- Installazione tramite **npm** (non è un *plugin Nx*)

```
npm i class-validator class-transformer
```

```
import { IsEmail, IsEnum, IsNotEmpty, IsString } from "class-validator";

export enum UserRole {
  USER = 'USER',
  ADMIN = 'ADMIN'
}

export class CreateUserDto {
  @IsString()
  @IsNotEmpty()
  name: string;

  @IsEmail()
  email: string;

  @IsEnum(UserRole, {
    message: 'Valid role required among USER or ADMIN'
  })
  role: UserRole;
};
```

Aggiunta di «class validators»



- Nel controller `users.controller.ts`

- Per aggiungere funzionalità di validazione avanzate
- Installazione tramite **npm** (non è un *plugin Nx*)
- Utilizzati con **ValidationPipe**

```
import { ValidationPipe } from '@nestjs/common';  
  
create(@Body(ValidationPipe) user: CreateUserDto) {  
  return this.userService.create(user);  
}
```


Built-in HTTP exceptions



- Per generare eccezioni «*off-the-shelf*» (si veda «**Exception Filters**» nella documentazione)

- Esempio (nel file **users.service.ts**)
 - Cercare un utente con un ID inesistente

```
import { NotFoundException } from '@nestjs/common';
```

```
getOneUser(id: number) {  
  const user = this.users.find(user => user.id === id);  
  
  if(!user) throw new NotFoundException("User not found!");  
  
  return user;  
}
```

Built-in HTTP exceptions



- Nella documentazione: docs.nestjs.com/pipes

INTRODUCTION

OVERVIEW

First steps
Controllers
Providers
Modules
Middleware
Exception filters
Pipes
Guards
Interceptors
Custom decorators

FUNDAMENTALS

TECHNIQUES

SECURITY

GRAPHQL

WEBSOCKETS

MICROSERVICES

NEW DEPLOYMENT

STANDALONE APPS

CLI

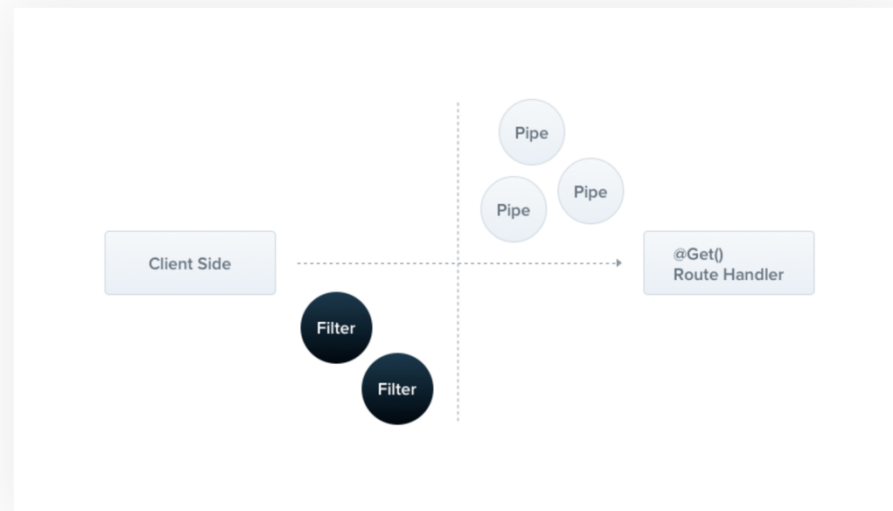
OPENAPI

RECIPES

FAQ

Exception filters

Nest comes with a built-in **exceptions layer** which is responsible for processing all unhandled exceptions across an application. When an exception is not handled by your application code, it is caught by this layer, which then automatically sends an appropriate user-friendly response.



Built-in HTTP exceptions



- Nella documentazione: docs.nestjs.org

INTRODUCTION

OVERVIEW ^

- First steps
- Controllers
- Providers
- Modules
- Middleware
- Exception filters**
- Pipes
- Guards
- Interceptors
- Custom decorators

FUNDAMENTALS v

TECHNIQUES v

SECURITY v

GRAPHQL v

WEBSOCKETS v

MICROSERVICES v

NEW **DEPLOYMENT**

STANDALONE APPS

CLI v

OPENAPI v

RECIPES v

FAQ v

Exception filters

Nest comes with a built-in **exceptions layer** which is responsible for catching exceptions thrown from application code, it is caught by this layer, which then automatically sends a response back to the client.

Client Side

Built-in HTTP exceptions

Nest provides a set of standard exceptions that inherit from the base `HttpException`. The most common HTTP exceptions:

- `BadRequestException`
- `UnauthorizedException`
- `NotFoundException`
- `ForbiddenException`
- `NotAcceptableException`
- `RequestTimeoutException`
- `ConflictException`
- `GoneException`
- `HttpVersionNotSupportedException`
- `PayloadTooLargeException`
- `UnsupportedMediaTypeException`
- `UnprocessableEntityException`
- `InternalServerErrorException`
- `NotImplementedException`
- `ImATeapotException`
- `MethodNotAllowedException`
- `BadGatewayException`
- `ServiceUnavailableException`
- `GatewayTimeoutException`
- `PreconditionFailedException`

Built-in HTTP exceptions



- Per generare eccezioni «*off-the-shelf*» (si veda «**Exception Filters**» nella documentazione)
- Secondo esempio (nel file **users.service.ts**)
 - Cercare nella lista degli utenti specificando un ruolo inesistente

```
getUsers(role?: "USER" | "ADMIN") {  
  if(role)  
  {  
    const rolesArray = this.users.filter(user => user.role === role);  
  
    if(rolesArray.length === 0) throw  
    new NotFoundException("User Role not valid");  
  
    return rolesArray;  
  }  
  return this.users;  
}
```

Link e riferimenti utili



- Link alla documentazione NestJS:
<https://docs.nestjs.com/>
- Link alla documentazione Swagger:
<https://swagger.io/doc>
- Dave Gray's tutorials on YouTube



Progettazione di applicazioni web full stack

Laboratorio di Node e NestJS

```
mirror_mod.use_x = False
mirror_mod.use_y = False
mirror_mod.use_z = True
operation = "MIRROR_Z"
mirror_mod.use_x = False
mirror_mod.use_y = False
mirror_mod.use_z = True

selection at the end -add
ob.select=1
ob.select=1
context.scene.objects.active
("Selected" + str(modifier))
mirror_ob.select = 0
bpy.context.selected_objects
data.objects[one.name].select
print("please select exactly

-- OPERATOR CLASSES -----

types.Operator):
X_mirror to the selected
X_mirror = mirror_x"
```



Prof. Devis Bianchini

Dip. di Università degli Studi di Brescia